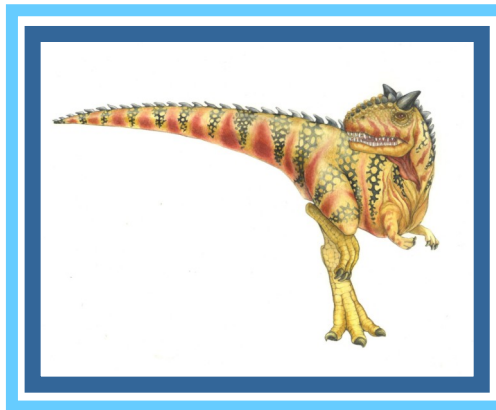


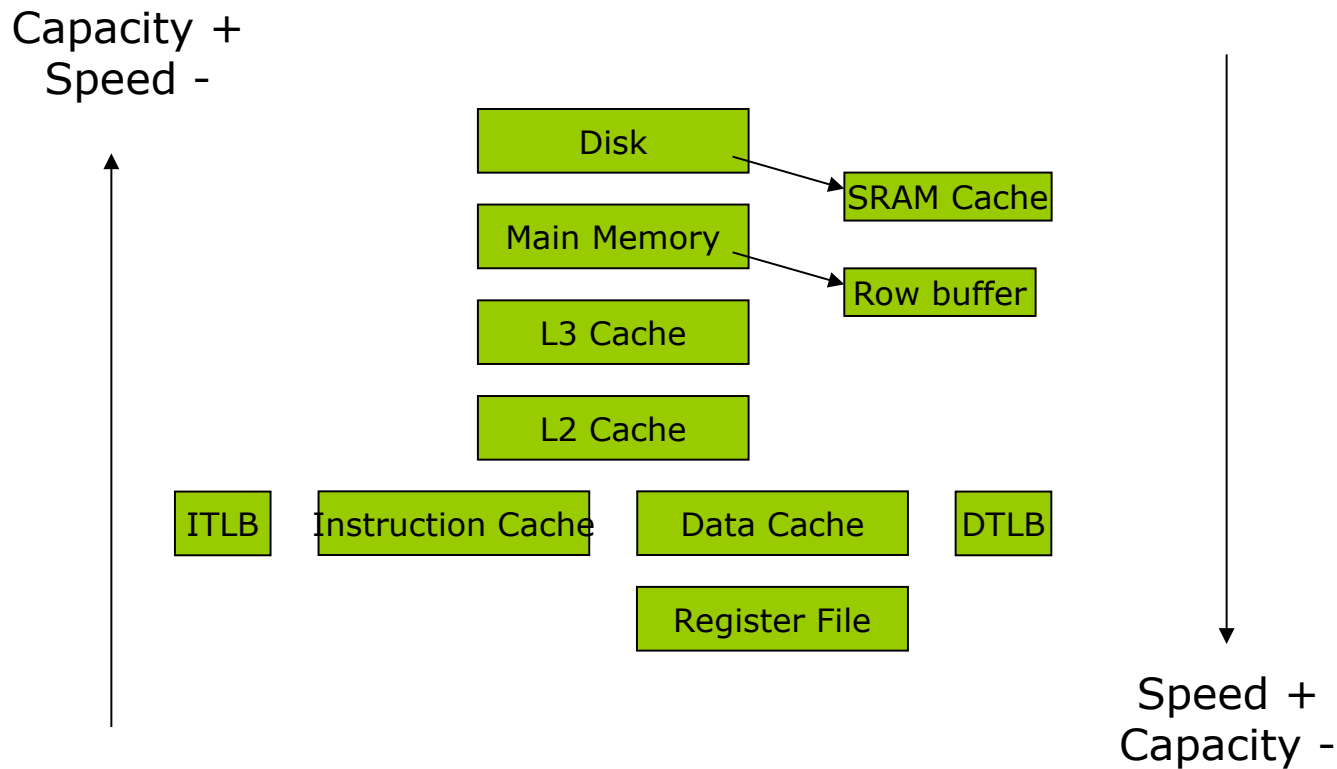
# Chapter 12: Cache

---





# Storage Hierarchy and Locality





# Memory Latency is *Long*

---

- 60-100ns not totally uncommon
- Quick back-of-the-envelope calculation:
  - 2GHz CPU
  - $\rightarrow 0.5\text{ns} / \text{cycle}$
  - 100ns memory  $\rightarrow 200$  cycle memory latency!
- Solution: Caches





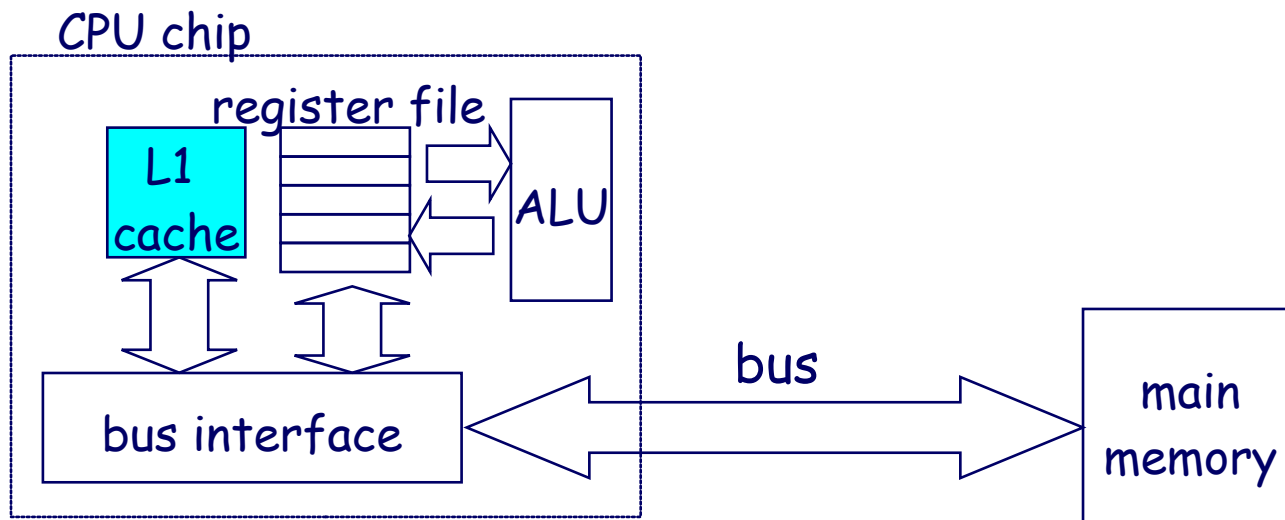
# Hardware cache memories

Cache memories are small, fast SRAM-based memories managed automatically in hardware

- Hold frequently accessed blocks of main memory

CPU looks first for data in L1, then in main memory

Typical system structure:





# Cache Basics

---

- Fast (but small) memory close to processor
- When data referenced
  - If in cache, use cache instead of memory
  - If not in cache, bring into cache  
(actually, bring entire **block** of data, too)
  - Maybe have to kick something else out to do it!

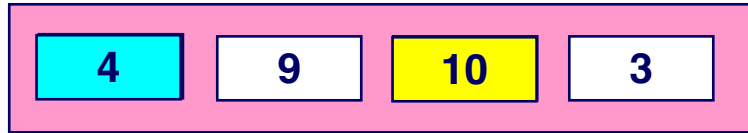
Key: Optimize the  
*average* memory  
access latency





# General cache mechanics

Cache:

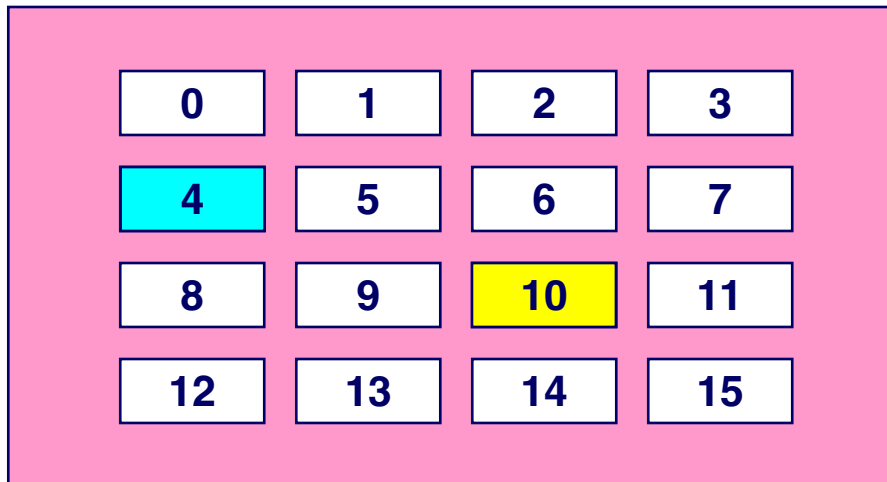


Smaller, faster, more expensive memory caches a subset of the blocks



Data is copied between levels in block-sized transfer units

Memory:



Larger, slower, cheaper memory is partitioned into “blocks”





# Cache Performance Metrics

---

## Miss Rate

- Fraction of memory references not found in cache (misses / accesses)
  - ▶  $1 - \text{hit rate}$  ☺
- Typical numbers (in percentages):
  - ▶ 3-10% for L1
  - ▶ can be quite small (e.g.,  $< 1\%$ ) for L2, depending on size, etc.

## Hit Time

- Time to deliver a line in the cache to the processor
  - ▶ includes time to determine whether the line is in the cache
- Typical numbers:
  - ▶ 1-2 clock cycle for L1
  - ▶ 5-20 clock cycles for L2

## Miss Penalty

- Additional time required because of a miss
  - ▶ typically 50-200 cycles for main memory (Trend: increasing!)





# Lets think about those numbers

---

Huge difference between a hit and a miss

- 100X, if just L1 and main memory

Would you believe 99% hits is twice as good as 97%?

- Consider these numbers:

cache hit time of 1 cycle

miss penalty of 100 cycles

So, average access time is:

97% hits:  $0.97 * 1 \text{ cycle} + 0.03 * 100 \text{ cycles} = \underline{4 \text{ cycles}}$

99% hits:  $0.99 * 1 \text{ cycle} + 0.01 * 100 \text{ cycles} = \underline{2 \text{ cycles}}$

Average/effective access time =

$\text{hit\_rate} * \text{cache\_latency} + (1 - \text{hit\_rate}) * \text{memory\_latency}$







# In Class Practice

---

- A CPU is equipped with a cache; Accessing a word takes 20 clock cycles if the data is not in the cache and 5 clock cycles if the data is in the cache.
- (1) What is the effective memory access time in clock cycles if the hit ratio is 90%.
- (2) What should be the hit ratio if the effective memory access time is 8 clock cycles.

□ (1) 6.5

□ (2) 80%





# Key Issues Related To Cache

---

## □ Important decisions

- Structure: what is the general organization of cache?
- Identification: how do we find a block in cache?
- Placement: where in the cache can a block go?
- Replacement: what to kick out to make room in cache?
  - ▶ LRU can also be applied here





# General Organization of a Cache

Cache is an array of sets

Each set contains one or more lines

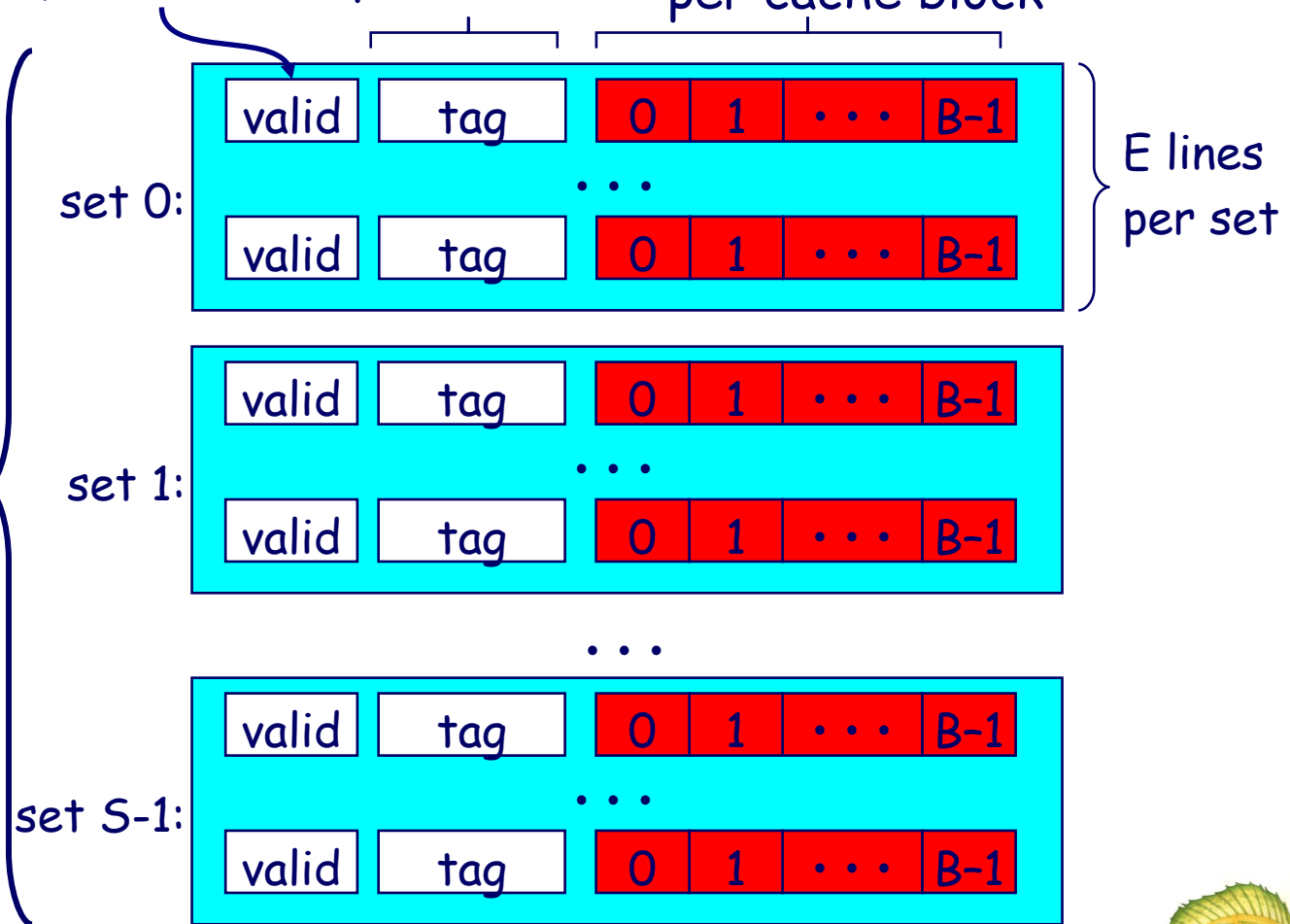
Each line holds a block of data

S sets

1 valid bit  
per line

+ tag bits  
per line

$B = 2^b$  bytes  
per cache block



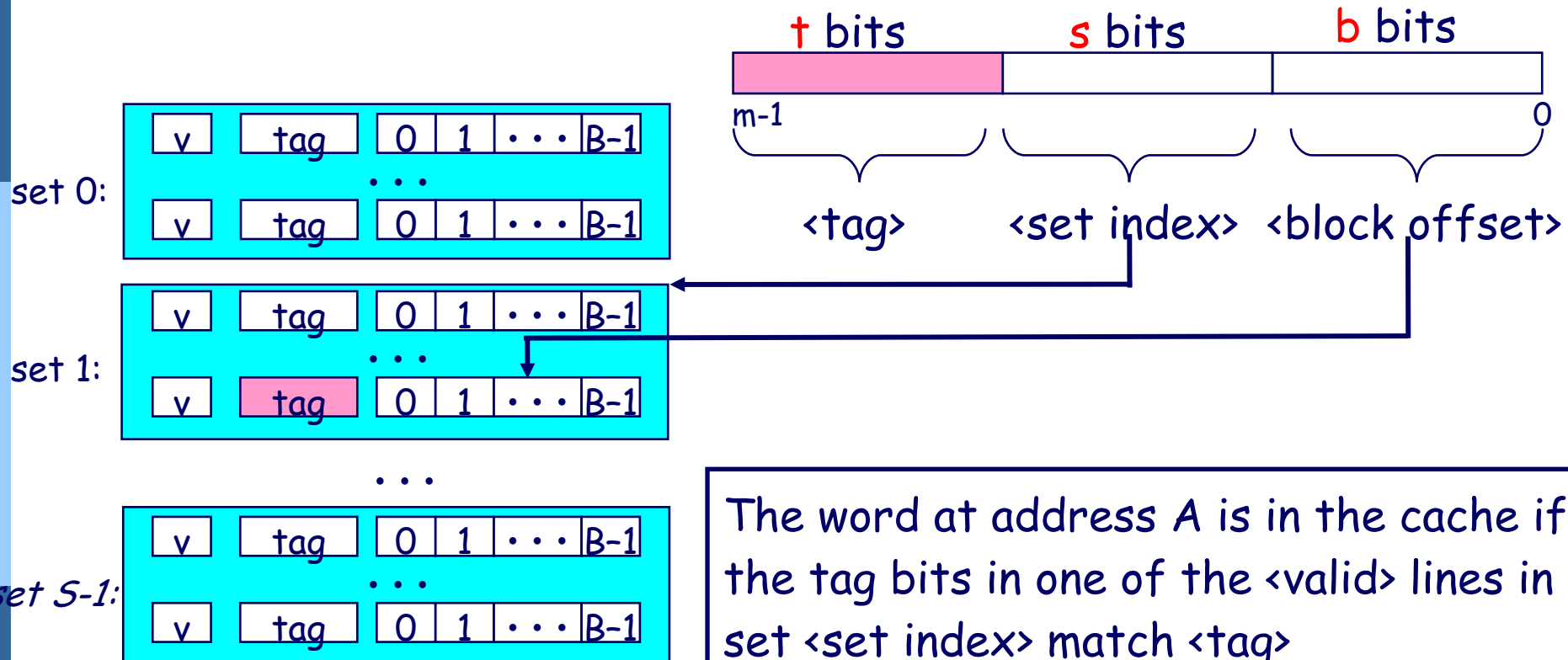
Cache size:  $C = B \times E \times S$  data bytes





# Addressing Caches

Address A:



The word at address A is in the cache if the tag bits in one of the <valid> lines in set <set index> match <tag>

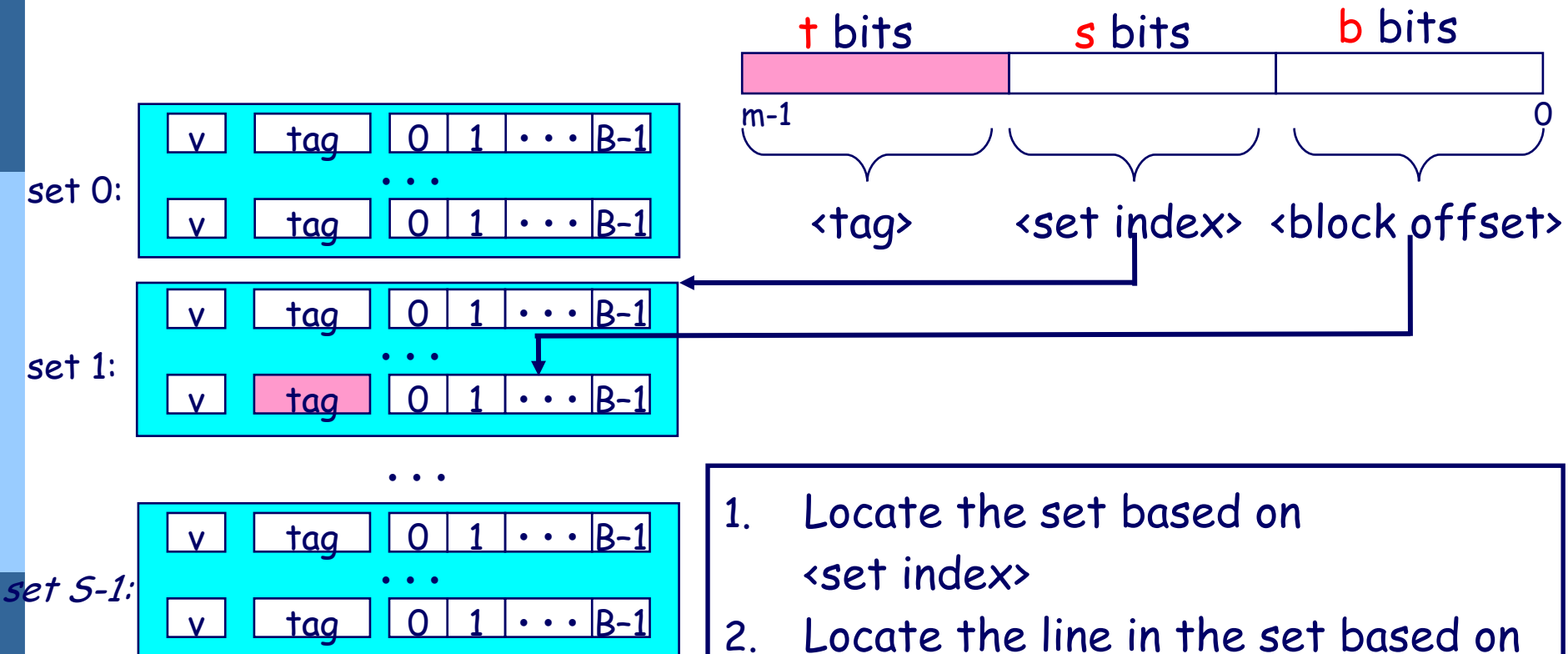
The word contents begin at offset <block offset> bytes from the beginning of the block





# Addressing Caches

Address A:



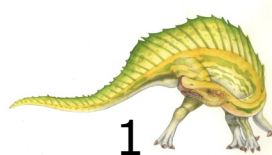
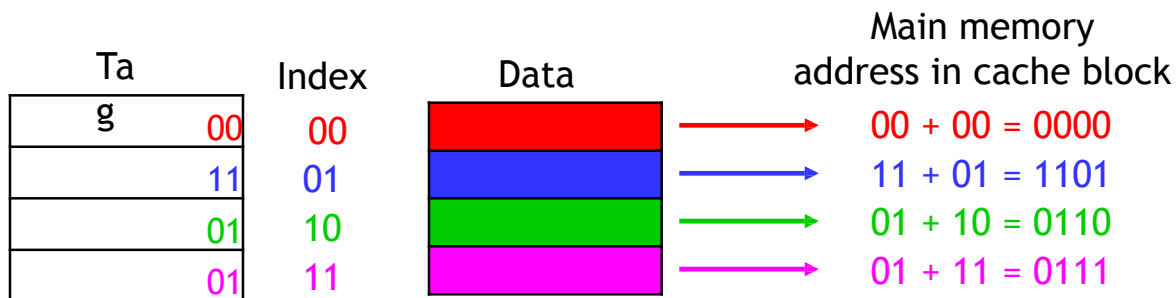
1. Locate the set based on <set index>
2. Locate the line in the set based on <tag>
3. Check that the line is valid
4. Locate the data in the line based on <block offset>





# Figuring out what's in the cache

- Set Index in the middle.
- Now we can tell exactly which addresses of main memory are stored in the cache, by concatenating the cache block tags with the block indices.

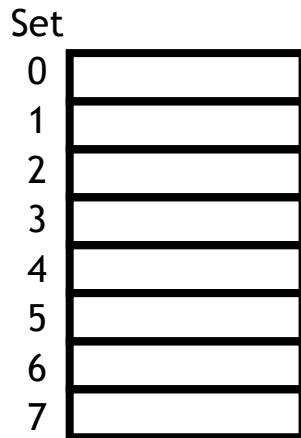




# Set associative caches are a general idea

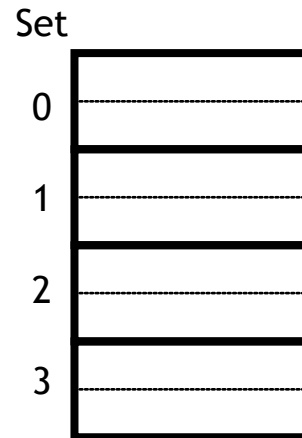
- By now you may have noticed the 1-way set associative cache is the same as a **direct-mapped** cache.
- Similarly, if a cache has  $2^k$  blocks, a  $2^k$ -way set associative cache would be the same as a **fully-associative** cache.

1-way  
8 sets,  
1 block each

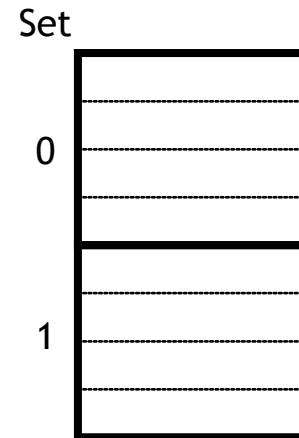


direct mapped

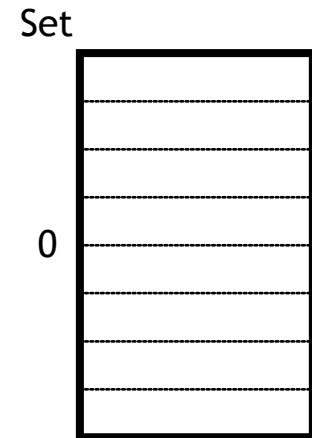
2-way  
4 sets,  
2 blocks each



4-way  
2 sets,  
4 blocks each



8-way  
1 set,  
8 blocks



fully associative

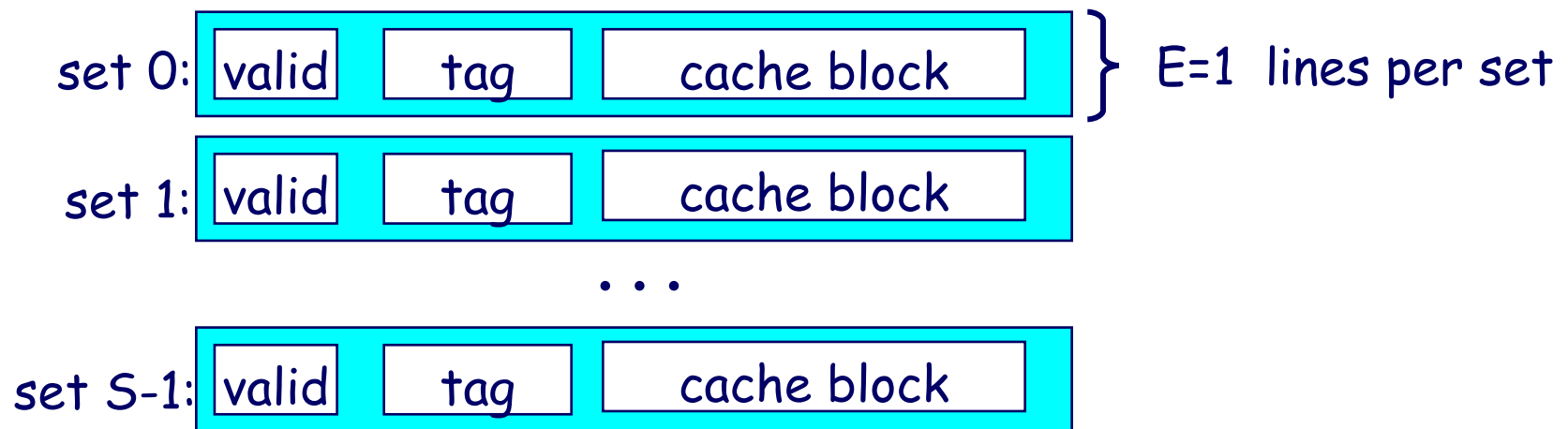




# Example: Direct-Mapped Cache

Simplest kind of cache, easy to build, and **fastest mapping**  
(only 1 tag compare required per access)

Characterized by exactly one line per set.







# Direct Mapping

- Which line to store the information?
  - Line ID = Block ID mod number of lines
- How many lines are there in a cache?
  - number of lines = cache size / words per line
  - E.g., cache size 2048 words, 16 words per line, there are 128 lines
  - Line bits =  $\log_2(\text{number of lines})$
  - Cache size:  $2^{(\text{line bits} + \text{word bits})}$
- How many tags are there?
  - number of tags = number of blocks / number of lines (or memory size / cache size)
  - Tag bits =  $\log_2(\text{number of tags})$
  - Memory size:  $2^{(\text{tag bits} + \text{line bits} + \text{word bits})}$

